Team No 67- (LLOYD Problem B)

1. Problem statement and our understanding: Many people want a more energy-efficient home and need a system to help them start. The challenge is to build an AI system that tells a customer how green their home is today, what they should improve, how much energy and carbon they could save, and whether the work could raise the value of the property.

We need a Green Home Score out of 100 built from seven components with fixed weights, a way to flag poorly performing homes and rank them by improvement potential, and a prioritised list of retrofits that are actually feasible for the home. It also gives savings in kWh per year for electricity and gas separately and carbon reduction from per-fuel emission factors. Property value impact is given as a range. The LLM is only allowed to explain these numbers and answer what-if questions after the engine has been rerun.

We see four difficulties. First, no public dataset links a home's features,with its metered energy use and the retrofit, so we have to generate labels ourselves, which have to follow realistic physics rather than being arbitrary. Second, retrofits interact with each other and must be accounted for. Third, the score has to be fair so we must not reward the same thing twice. Fourth, results on synthetic labels do not prove real-world accuracy, the value uplift is only indicative, and when data is missing or conflicting the system should say so instead of guessing.

2. Idea: Our idea is a physics-based home simulator paired with machine-learning models with deterministic scoring and carbon calculations, with the LLM kept strictly as an explainer. It creates realistic labels and reruns scenarios. LightGBM models learn from noisy data and give calibrated ranges. When a customer enters real energy usage, we calibrate the simulator to their home so the starting figures and the projected state reconcile. We also separate gross consumption from purchased energy. The score uses gross consumption so solar is not rewarded twice, while improvement potential, bills and carbon use purchased energy. First the customer sees every feasible retrofit in a sensible order, then the list is re-ranked around their goal, budget and choices. Every LLM reply is checked so that any number init must come from the engine's output.

3. High-level approach, pipeline or architecture

a. **Input:** Customer enters basic property details, energy use, and their goal/budget. Unknowns are flagged rather than guessed, with a confidence level for the result.
b. **Data:** We generate ~30,000 realistic synthetic homes using a physics-based simulator, calibrated against our data.
c. **Pipeline:** Validate inputs → calculate Green Home Score → establish baseline energy → identify issues → filter and rank feasible retrofits → simulate retrofit combinations → calculate savings, bills, carbon and value → LLM explains the results.

d. **Score:** A 0-100 score combines the seven required areas with the given weights. The dashboard shows the component-level contributions, and the same scoring method is used before and after retrofits.
e. **Retrofits:** Unsuitable options are removed first. Feasible options are ranked by suitability, cost and impact. Combinations are simulated together so interactions are captured, and results are cached for quick what-if changes.
f. **Models:** LightGBM estimates electricity/gas use with uncertainty ranges. Carbon is calculated directly from per-fuel savings and emission factors. Property value is given as an indicative range, not a guaranteed or causal uplift.
g. **LLM:** The LLM only explains results. What-if questions are rerun through the calculation engine first, and a verifier checks that the LLM does not invent numbers. Gemini is the current API, with a free/local fallback for the MVP.
h. **Evaluation:** We test prediction, retrofit feasibility, savings, value, ranking and LLM groundedness, including missing data, conflicts, infeasible/interacting retrofits and heat-pump cases.
i. **Stack:** Python, LightGBM/scikit-learn, FastAPI, Streamlit/React, Plotly and Docker.

4. Planned testing methodology

Splits. We will split by customer or property, and use about 70% to train, 15% for validation and 15% for testing, so the same home never appears in two splits. We also plan to hold out one region and one age band as a harder test, and we will exclude any field that would not be known at prediction time.

Baselines and metrics. Each model will be compared with a simple baseline on the same split. Feasibility will be judged by precision and recall. Savings will be compared with the simulator's reference values, and the value model against average prices by region and type, with interval coverage. We also test with and without physics features and residual learning to show what actually helps.

Engine checks. We will test that solar, battery, grid import and export balance exactly, and make sure that no energy figure goes negative. We will also make sure that every subscore only improves as the home improves.

Scenario tests. We will test edge cases including missing or conflicting data, infeasible or interacting retrofits, electricity rising after a heat pump, and insufficient information to estimate savings or value. The system should flag the estimate, explain it clearly, or decline to answer.

LLM evaluation. We will measure how often the verifier passes, audit groundedness and factual accuracy and track how often the template fallback is needed.

Reproducibility. We are planning to use fixed seeds, clear instructions to run code,synthetic data generated and label rules, so the whole pipeline can be rerun end to end.